Reviewer Pack — Schatten p-Norm Inverse Proportional to Disjointness Communi…

Entry cd35d4c70022 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 14:59:47 UTC

Schatten p-Norm Inverse Proportional to Disjointness Communication Complexity

Entry ID: `cd35d4c70022` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-11 14:59:47 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative L^p Geometry **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

> For a random DISJOINTNESS instance M_n over n-bit inputs, the Schatten p-norm ||M_n||_p satisfies ||M_n||_p = Θ(1/n^{1/p}) for p ∈ (1,2]. This holds for all n ≤ 40 with 95% confidence across 30 random seeds.

Rationale (proposer's reasoning):

> Schatten norms quantify operator regularity in noncommutative spaces, which may mirror the information-theoretic constraints of distributed computation. The power-law scaling suggests a deep connection between matrix geometry and communication lower bounds.

Taxonomy category: `COMM_DISJ` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `8df7cf54927f049d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def generate_disjointness_matrix(n):
    M = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            if random.choice([True, False]):
                M[i][j] = 1
                M[j][i] = 1
    return M

def matrix_multiplication(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def trace(matrix):
    n = len(matrix)
    tr = 0
    for i in range(n):
        tr += matrix[i][i]
    return tr

def schatten_p_norm(M, p):
    n = len(M)
    A = matrix_multiplication(M, M)
    det_A = 1.0
    for i in range(n):
        det_A *= A[i][i]
    if det_A <= 0:
        return float('inf')
    return (det_A ** (1 / p)) * n ** (-1 / p)

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```
    n = 40
    M = generate_disjointness_matrix(n)
    p_values = [1.5, 1.75, 1.9]
    C_estimates = []
    for p in p_values:
        norm = schatten_p_norm(M, p)
        C_estimates.append(norm * n ** (1 / p))

    # Fit regression model
    from scipy.stats import linregress
    x = [1 / n ** (1 / p) for p in p_values]
    y = C_estimates
    slope, intercept, r_value, p_value, std_err = linregress(x, y)

    if abs(slope - 1.2) > 0.2:
        return {
            "metric_name": "Schatten p-norm",
            "metric_value": None,
            "instances_tested": len(p_values),
            "conjecture_holds": False,
            "counterexample": f"Slope {slope} does not match expected 1.2"
        }

    return {
        "metric_name": "Schatten p-norm",
        "metric_value": slope,
        "instances_tested": len(p_values),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    supported_count = sum(1 for r in results if r["conjecture_holds"])
    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results)

    if supported_count >= 0.8 * len(seeds):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={supported_cour
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Slope does not match expected 1.2\" first_faili
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_81bdb183.py", line 92, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_81bdb183.py", line 64, in run_trial
    from scipy.stats import linregress
ModuleNotFoundError: No module named 'scipy'
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed due to missing scipy dependency, preventing verification of the conjecture's validity | next: Install scipy and re-run tests with sufficient seeds to validate the conjecture

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111339
2	propose	ollama_remote	qwen3:8b	0	0	52833
3	preregistration	ollama_remote	qwen3:8b	0	0	24372
4	novelty	ollama_remote	qwen3:8b	0	0	20839
5	novelty	ollama_remote	qwen3:8b	0	0	15880
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12986
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9621
8	judge	ollama_remote	qwen3:8b	0	0	14846

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 262717 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/cd35d4c70022.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cd35d4c70022.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/cd35d4c70022.tar.gz` (if generated)